Laboratorio SSL — SRDSI

Jorge Arévalo Jon González

Octubre 2026

1. Introducción y objetivo

El objetivo del laboratorio es doble: (1) proteger con OpenSSL el protocolo de aplicación `cli-infouser`/`serv-infouser` —que hasta ahora viajaba en claro sobre TCP— y (2) analizar las alertas TLS que se generan ante dos situaciones de error con los certificados.

La aplicación original (Ejercicio 1 del enunciado) permite al administrador obtener la lista de usuarios conectados: el cliente pide la información, el servidor ejecuta `w`, la guarda en un fichero temporal y la envía al cliente, que la almacena en el fichero indicado como segundo parámetro.

Tareas realizadas:

1. Migrar el cliente y el servidor a la API de sockets seguros de OpenSSL.
2. Reproducir y capturar dos escenarios de error: certificado inexistente y certificado firmado por una CA que el cliente no reconoce.
3. Analizar las alertas TLS observadas.
4. Contabilizar el tiempo invertido y valorar la actividad.

Todo el código y los scripts de captura están en el repositorio (`lab-ssl/source_files/` y `lab-ssl/run-lab.sh`), y la evidencia en `lab-ssl/capturas/`.

2. Ejercicio 1 — Del protocolo TCP a OpenSSL

2.1. Protocolo de aplicación

El protocolo se mantiene idéntico al original; sólo cambia el transporte. El servidor escucha en el puerto fijo 1064 (`IPPORT_RESERVED + 40`), ejecuta `w > /tmp/file_users` y envía al cliente una cabecera `OK:<tamaño>` seguida del contenido del fichero por bloques. El cliente valida la cabecera y vuelca los bytes recibidos en el fichero local.

En la versión segura, las mismas operaciones se hacen con `SSL_accept`/ `SSL_connect`, `SSL_read` y `SSL_write` sobre el socket ya establecido.

2.2. Servidor

El servidor inicializa el contexto TLS, carga su certificado y su clave privada y comprueba que se corresponden:

```
method = TLS_server_method();
ctx = SSL_CTX_new(method);
CHK_NULL(ctx);
```

Jorge Arévalo
Jon González

jarevalo004@ikasle.ehu.eus
jgonzalez363@ikasle.ehu.eus
http://www.sc.ehu.es/isg

GIPUZKOAKO CAMPUSA
CAMPUS DE GUIPUZCOA
Paseo Manuel de Lardizabal, 1
20018 Donostia (Gipuzkoa)

```
err = SSL_CTX_use_certificate_file(ctx, CERTFILE, SSL_FILETYPE_PEM);
CHK_SSL(err);
err = SSL_CTX_use_PrivateKey_file(ctx, KEYFILE, SSL_FILETYPE_PEM);
CHK_SSL(err);
err = SSL_CTX_check_private_key(ctx);
CHK_ERR(err,"La clave privada no corresponde al certificado\n");
```

Como el cliente **no presenta certificado**, el servidor no carga la CA del cliente ni exige verificación mutua (no se llaman `SSL_CTX_load_verify_locations` ni `SSL_CTX_set_verify`). Se añade además `SO_REUSEADDR` para poder reiniciar el servidor sin que lo impidan sockets en `TIME_WAIT`.

El bucle de servicio sustituye el `accept` plano por el saludo TLS:

```
do {
    c = accept(s, NULL, 0);
    ssl = SSL_new(ctx);
    SSL_set_fd(ssl, c);
    err = SSL_accept(ssl);          /* handshake TLS */
    enviar_usuarios(ssl);            /* SSL_write por bloques */
    SSL_shutdown(ssl);
    SSL_free(ssl);
    close(c);
} while (1);
```

2.3. Cliente

El cliente sólo necesita confiar en la CA del servidor. No carga certificado de cliente:

```
method = TLS_client_method();
ctx = SSL_CTX_new(method);

SSL_CTX_set_cipher_list(ctx, "AES128-SHA256:AES256-CCM:AES256-GCM-SHA384");
SSL_CTX_set_max_proto_version(ctx, TLS1_2_VERSION);   /* descifrable en Wireshark */

SSL_CTX_load_verify_locations(ctx, CA_CERT, NULL);    /* certificados/s-ca_cert.pem */
SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
SSL_CTX_set_verify_depth(ctx, 1);

SSL_CTX_set_keylog_callback(ctx, key_log_cb);         /* exporta secretos TLS */
```

La fijación de TLS 1.2 y de los cifrados es imprescindible para poder descifrar las tramas en Wireshark (que sólo descifra TLSv1.2 con un *keylog file*). El callback `key_log_cb` añade cada secreto a `keylog-ssl.log`.

2.4. Compilación y prueba

```
$ gcc -Wall -O2 -o serv-infouser serv-infouser.c -lssl -lcrypto
$ gcc -Wall -O2 -o cli-infouser  cli-infouser.c  -lssl -lcrypto
$ ./serv-infouser &
$ ./cli-infouser 127.0.0.1 /tmp/usuarios.txt
Conectado con cifrado AES128-SHA256
Verificacion del servidor: OK
```

Al terminar, el fichero recibido coincide con la salida de `w` que generó el servidor (mismo tamaño y contenido; en la captura de referencia, 221 bytes).

3. Ejercicio 2 — Análisis de las alertas TLS

3.1. Montaje de las capturas

El tráfico se captura en la interfaz de *loopback* de la máquina virtual con `dumpcap -i lo -w captura.pcapng`, y se analiza con `tshark`/Wireshark cargando `keylog-ssl.log` en *Edit → Preferences → Protocols → TLS → (Pre)-Master-Secret log filename*. El script `lab-ssl/run-lab.sh` automatiza la compilación, la reproducción de los tres escenarios y el análisis.

Los certificados empleados son los del repositorio, sustituidos el 2026-10-19: el certificado de servidor (`s_cert.pem`, serial `865EE37EE2F16ABE`) y el de cliente (`c_cert.pem`, serial `16B61DD2792713CF`) son válidos del 2026-10-19 al 2037-10-01, igual que las CAs `CA_SERV-SRDSI` y `CA_CLI-SRDSI`. El certificado del servidor extraído de `01_ok.pcapng` coincide con `s_cert.pem`.

3.2. Escenario correcto (referencia)

En el caso correcto el handshake se completa y la información viaja cifrada pero se puede descifrar con el *keylog*:

```
4   TLSv1    Client Hello
6   TLSv1.2  Server Hello, Certificate, Server Hello Done
8   TLSv1.2  Client Key Exchange, Change Cipher Spec, Finished
9   TLSv1.2  New Session Ticket, Change Cipher Spec, Finished
10  TLSv1.2  Application Data, Application Data, Alert (Close Notify)
```

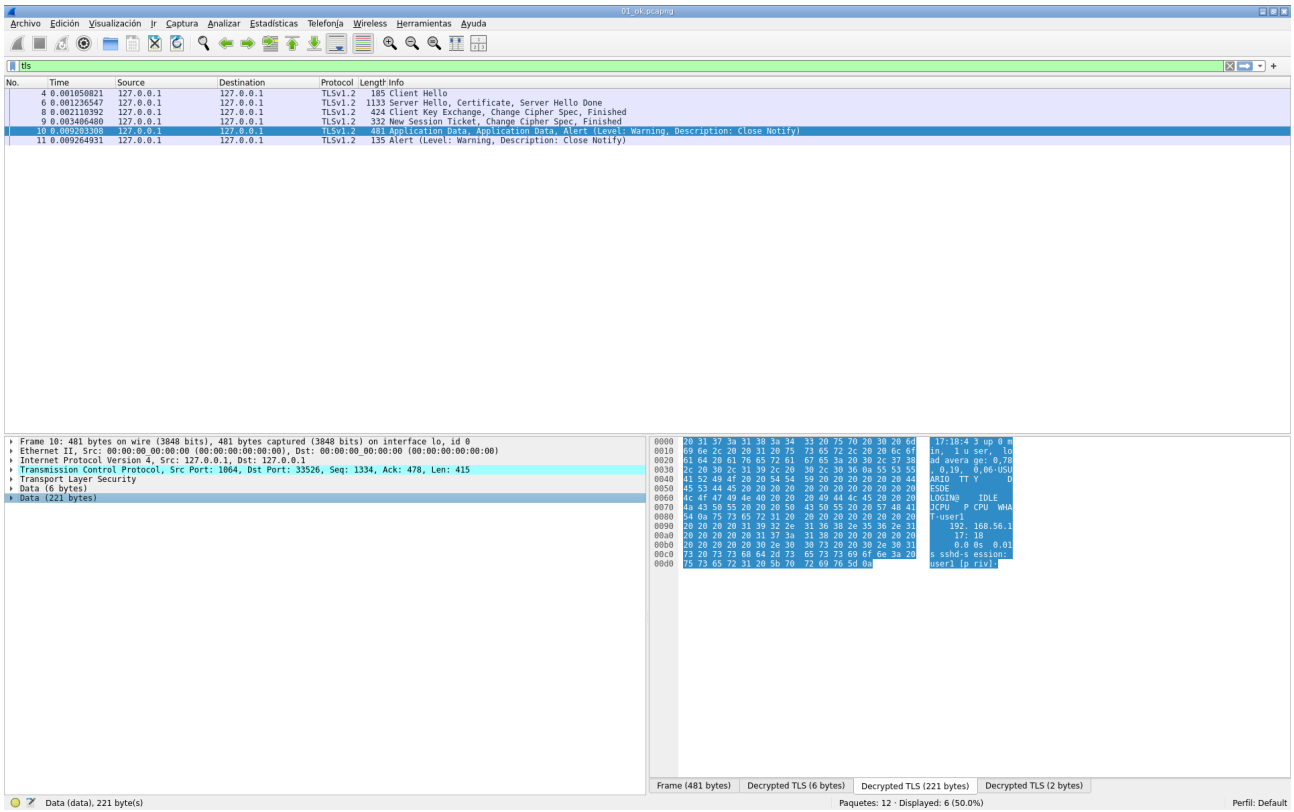


Figura 1: Handshake TLS correcto. Con el *keylog* cargado, el frame de *Application Data* seleccionado muestra la capa TLS descifrada (`Data (6 bytes)` y `Data (221 bytes)`) y, al seleccionar la fuente «Decrypted TLS» en el panel de bytes, se lee el contenido en claro (la salida de `w`).

El contenido descifrado se puede volcar completo con `Follow → TLS Stream` (o `tshark -z follow,tls,ascii,0`), donde se reconoce el protocolo de la aplicación (`OK:<tamaño>`) seguido de la salida de `w`:

```
$ tshark -r 01_ok.pcapng -o tls.keylog_file:01_ok.keylog -q -z follow,tls,ascii,0
Filter: tls.stream eq 0
Node 0: 127.0.0.1:33526
Node 1: 127.0.0.1:1064
        6
OK:221
        221
 17:18:43 up 0 min,  1 user,  load average: 0,78, 0,19, 0,06
USUARIO  TTY      DESDE            LOGIN@   IDLE   JCPU   PCPU  WHAT
user1             192.168.56.1     17:18           0.00s  0.01s sshd-session: user1 [priv]
```

3.3. Caso 1 — Certificado inexistente

Se elimina (o renombra) `certificados/s_cert.pem` y se arranca el servidor. Al no disponer de certificado, el servidor no puede negociar ninguno de los cifrados fijados por el cliente (todos usan intercambio de clave RSA) y responde al `Client Hello` con una **alerta fatal `handshake_failure` (40)**:

```
4   TLSv1    Client Hello
6   TLSv1.2  Alert (Level: Fatal, Description: Handshake Failure)
```

En el servidor se observa el error `no shared cipher` y en el cliente `ssl/tls alert handshake failure: SSL alert number 40`.

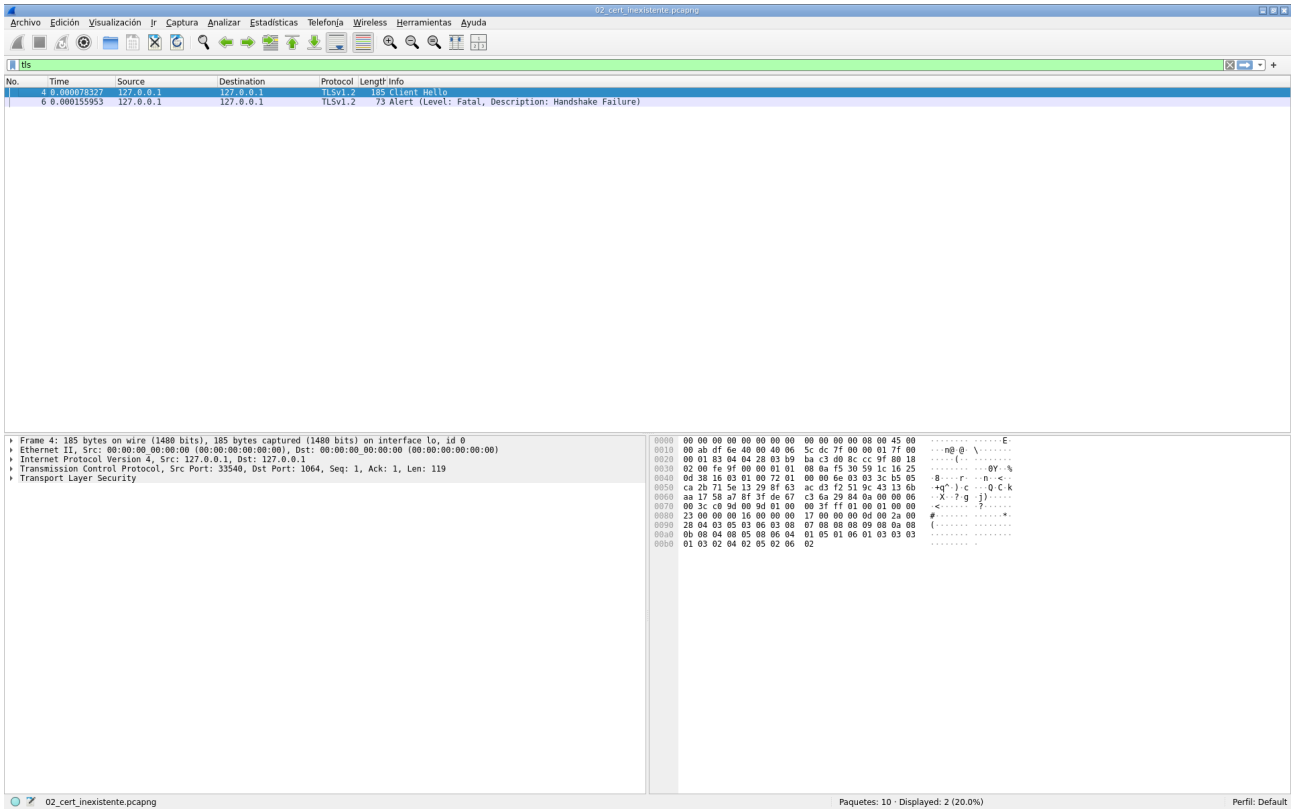


Figura 2: Certificado inexistente: el servidor responde Handshake Failure (40).

3.4. Caso 2 — CA desconocida

Se configura el cliente para que confíe en la CA equivocada (`certificados/c-ca_cert.pem` en vez de `s-ca_cert.pem`). El handshake avanza hasta que el servidor envía su `Certificate`; el cliente no puede construir una cadena de confianza y aborta con una **alerta fatal `unknown_ca` (48)**:

```
4   TLSv1    Client Hello
6   TLSv1.2  Server Hello, Certificate, Server Hello Done
8   TLSv1.2  Alert (Level: Fatal, Description: Unknown CA)
```

El cliente registra `certificate verify failed` y el servidor recibe `tlsv1 alert unknown ca: SSL alert number 48`.

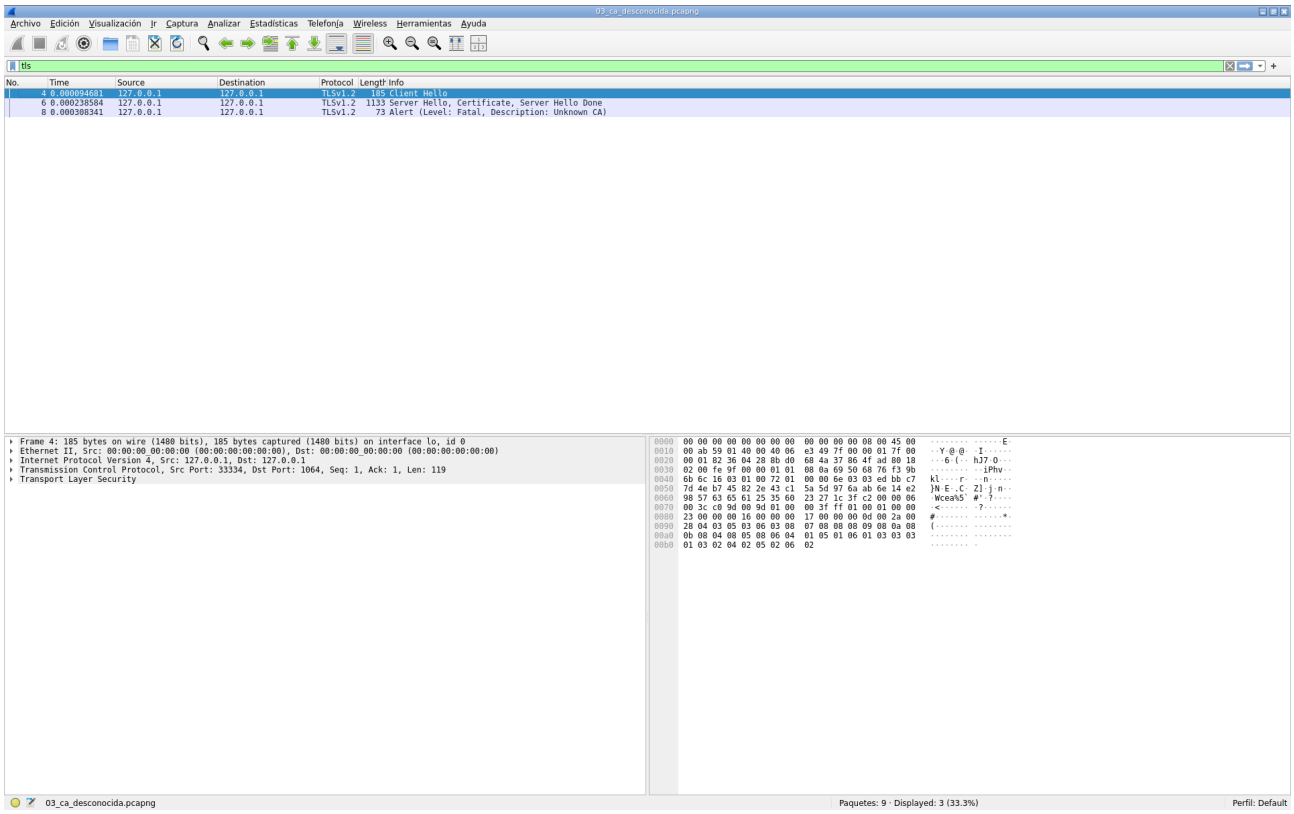


Figura 3: CA desconocida: el cliente responde Unknown CA (48).

3.5. Resumen de alertas

Situación	Origen de la alerta	Alerta TLS
CA correcta	—	handshake correcto
Certificado inexistente	servidor	`handshake_failure` (40, fatal)
CA no reconocida	cliente	`unknown_ca` (48, fatal)

4. Valoración de la actividad

La actividad resulta muy didáctica: partir de un protocolo TCP en claro y hacerlo seguro obliga a entender el ciclo de vida de un contexto SSL (`SSL_CTX`), la carga de certificados y el papel de la verificación. El Ejercicio 2 es

especialmente útil porque muestra en Wireshark la diferencia entre un fallo de negociación (el servidor ni siquiera puede completar el saludo) y un fallo de confianza (el cliente rechaza el certificado del servidor), con alertas distintas en cada caso. La necesidad de fijar TLS 1.2 y exportar el *keylog* para descifrar refuerza la comprensión de las claves de sesión y de la pila TLS. Como aspecto mejorable, la máquina virtual no traía `libssl-dev`, que hubo que instalar (`apt-get install libssl-dev`) antes de poder compilar el código.

5. Tiempo dedicado

Tarea	Responsable	Tiempo
Estudio del enunciado y del material de teoría	Jorge y Jon	0,5 h
Ejercicio 1: migración del cliente y servidor a OpenSSL	Jorge y Jon	2,0 h
Ejercicio 2: escenarios de error y capturas	Jon	1,5 h
Análisis de las alertas y redacción del informe	Jorge	1,5 h
Total		**5,5 h**

La teoría y la migración se realizaron conjuntamente. Reparto resultante: 2,75 h Jorge y 2,75 h Jon (0,25 + 1,0 h de las tareas conjuntas, más 1,5 h de su parte).

6. Referencias

1. Enunciado del laboratorio: `Lab-ssl-actividad.pdf` y `Lab-ssl-teoria.pdf`.
2. OpenSSL Documentation. https://www.openssl.org/docs/
3. Wireshark — TLS decryption (keylog file). https://wiki.wireshark.org/TLS